

Roteiro de Estudo — POO / Java

Exercícios de provas anteriores + simulado oficial do professor
Compilado para revisão antes da prova de 17/set/2026

Prova 1 — Classe ReservaDeHotel

Fonte: prova de anos anteriores (foto) — data parcialmente legível (~26/set/202_)

Classes já implementadas (dadas como prontas)

Data

- Construtor: `public Data(byte dia, byte mes, short ano)`
- Getters e setters para todos os atributos
- `public void avanceUmDia()` — avança um dia
- `public void retrocedaUmDia()` — retrocede um dia
- Dos métodos `toString`, `equals`, `hashCode`, `clone`, construtor de cópia e `compareTo`, aqueles que fizerem sentido, dados os métodos que a classe tem

Moeda

- Construtor: `public Moeda(long centavos, String codigo)` — cria valor a partir de centavos e código da moeda (BRL, USD, EUR, etc.)
- `public Moeda converterPara(String novaMoeda, double taxaConversao)` — retorna nova moeda convertida
- `public Moeda somar(Moeda outra)` — retorna a soma, se mesma moeda
- `public Moeda multiplicar(int fator)` — retorna o produto
- Dos métodos `toString`, `equals`, `hashCode`, `clone`, construtor de cópia e `compareTo`, aqueles que fizerem sentido

O que é PEDIDO: classe ReservaDeHotel

Representa uma reserva de hotel com data de entrada, data de saída e valor da diária em uma dada moeda.

Atributos obrigatórios:

- Data entrada
- Data saída
- Moeda diaria

Restrições:

- saída deve ser posterior a entrada

Métodos a implementar:

- 1 **Construtor:** `public ReservaDeHotel(Data entrada, Data saida, Moeda diaria)` — deve validar a restrição.
- 2 **getValorTotal:** `public Moeda getValorTotal()` — retorna o valor total = diária × número de diárias.
- 3 **converterValorTotal:** `public Moeda converterValorTotal(String novaMoeda, double taxa)` — retorna valor total convertido para nova moeda.
- 4 **adiaSaida:** `public void adiaSaida(Data nova)` — adia a saída para a nova data informada, que necessariamente deve ser posterior à atual data de saída, ou uma exceção deve ser lançada. Se a nova data estiver ENTRE a atual entrada e a atual saída → exceção com texto "Anterior à atual"

saída". Se a nova data estiver ANTES da atual entrada → exceção com texto "Anterior à atual entrada".

- 5 Dos métodos toString, equals, hashCode, construtor de cópia, clone e compareTo, aqueles que você julgar necessários.

Prova 2 — Classe PartidaDeFutebol

Fonte: prova de anos anteriores (foto) — Prof. André Carvalho — 17/set/2024

Classes já implementadas (dadas como prontas)

Data — igual à da Prova 1 (construtor, avanceUmDia, retrocedaUmDia, equals, toString → "dd/mm/aaaa").

Placar — no papel o título impresso diz "Horario", mas foi corrigido à mão para "Placar"; os métodos deixam claro que é um placar de partida, não um horário.

- Construtor: `public Placar(String timeA, String timeB)`
- `public void marcarGol(String time)` — registra gol
- `public void encerrar()` — encerra a partida
- `public String getVencedor()` — retorna vencedor ou null
- `public String toString()` — ex: "Flamengo 3 x 2 Corinthians"

O que é PEDIDO: classe PartidaDeFutebol

Representa uma partida de futebol, definida por sua data e placar.

Atributos obrigatórios:

- Data data
- Placar placar

Métodos a implementar:

- 1 **Construtor**: `public PartidaDeFutebol(Data data, String timeA, String timeB)` — inicializa placar em 0x0.
- 2 **registrarGol**: `public void registrarGol(String time)` — delegar ao placar.
- 3 **encerrar**: `public void encerrar()` — delegar ao placar.
- 4 **getVencedor**: `public String getVencedor()` — retorna vencedor ou null.
- 5 Dos métodos `toString`, `equals`, `hashCode`, construtor de cópia, `clone` e `compareTo`, aqueles que você julgar necessários.

Prova 3 — Classe Evento

Fonte: prova de anos anteriores (foto) — Prof. André Carvalho — 17/set/2024

Classes já implementadas (dadas como prontas)

Data (mesma de sempre) e **Horario** — a definição completa da classe Horario não aparece nas fotos enviadas; só o enunciado do Evento, que a referencia.

O que é PEDIDO: classe Evento

Representa um compromisso que ocorre em uma determinada data e dura de um horário de início até um horário de fim.

Atributos obrigatórios:

- Data data — a data do evento
- Horario inicio — horário de início
- Horario fim — horário de término

Restrições:

- O horário de término não pode ser anterior ao horário de início.

Métodos a implementar:

- 1 **Construtor**: `public Evento(Data data, Horario inicio, Horario fim)` — cria o evento validando que fim não é anterior a inicio. Se a condição não for respeitada, lançar exceção.
- 2 **getDuracaoEmMinutos**: `public int getDuracaoEmMinutos()` — calcula a diferença entre fim e inicio, convertida em minutos.
- 3 **contem**: `public boolean contem(Horario h)` — retorna true se h está entre inicio e fim (inclusive as bordas), senão false.
- 4 **colide**: `public boolean colide(Evento outro)` — retorna true se os dois eventos acontecem na mesma data e possuem interseção de horários.
- 5 **getIntersecao**: `public Evento getIntersecao(Evento outro)` — retorna novo Evento representando a interseção dos dois, ou null se não houver. inicio = o mais tarde entre os dois inícios; fim = o mais cedo entre os dois terminos.
- 6 Dos métodos `toString`, `equals`, `hashCode`, construtor de cópia, `clone` e `compareTo`, aqueles que você julgar necessários.

Exercício 4 — Classe Reserva

Fonte: relato de colega de outra turma (2ª mão) — reconstituído a partir da descrição enviada por ele

Este enunciado é de relato de terceiros (não é uma foto da prova). Os itens marcados abaixo como "não confirmados" são inferências coerentes com o padrão das outras provas, não fatos garantidos.

Classes já implementadas (dadas como prontas)

Data e Horário — ambas já possuem métodos prontos para comparação entre si.

O que é PEDIDO: classe Reserva

"A partir dessa classe, você deve implementá-la seguindo as regras de negócio abaixo, definindo por conta própria quais métodos são necessários para atendê-las corretamente."

Atributos (não confirmados com exatidão):

- qtdPessoas (quantidade de pessoas)
- valor (valor da hospedagem)
- um código (usado para igualdade — regra 4)
- algum atributo de status (confirmada / cancelada)
- provavelmente Data e/ou Horário como atributos (não citado explicitamente, mas coerente com o padrão das outras provas)

Regras de negócio:

- 1 A quantidade de pessoas e o valor da hospedagem não podem ser negativos.
- 2 Uma reserva cancelada não pode ser confirmada.
- 3 Uma reserva cancelada não pode ter sua quantidade de pessoas alterada.
- 4 Duas reservas são consideradas iguais quando possuem o mesmo código.

Métodos obrigatórios:

- cancelar()
- confirmar()

(ambos respeitando as regras acima)

Demais métodos: a critério — identificar e implementar o que for necessário para a classe funcionar corretamente e seguir boas práticas de OO (o enunciado sugere fortemente: construtor com validação, getters/setters com validação onde fizer sentido, e provavelmente equals/hashCode por causa da regra do código).

Simulado Oficial — Classe Complexo

Fonte: PDF do professor — Simulado da 1ª Prova, PPOO, 2º Semestre de 2026 — Profs. André Carvalho & J.G. Pícolo — 14/set/2026 — Tempo: 80 min. O professor indicou que a prova real terá este nível de dificuldade.

Contexto (base teórica fornecida)

Números complexos são números da forma $a+bi$, onde i representa a raiz quadrada de -1 , a é o coeficiente real e b o coeficiente imaginário.

- **Soma:** $(a_1+b_1i) + (a_2+b_2i) = (a+b)i$, onde $a = a_1+a_2$ e $b = b_1+b_2$
- **Diferença:** $(a_1+b_1i) - (a_2+b_2i) = (a+b)i$, onde $a = a_1-a_2$ e $b = b_1-b_2$
- **Multiplicação:** $(a_1+b_1i) \times (a_2+b_2i) = (a+b)i$, onde $a = a_1a_2 - b_1b_2$ e $b = a_1b_2 + a_2b_1$
- **Divisão:** $(a_1+b_1i) \div (a_2+b_2i) = (a+b)i$, onde $a = (a_1a_2 + b_1b_2)/(a_2^2 + b_2^2)$ e $b = (a_2b_1 - a_1b_2)/(a_2^2 + b_2^2)$

O que é PEDIDO: classe Complexo

- 1 **Estrutura interna** — declarar os atributos necessários para representar uma instância (coeficiente real e coeficiente imaginário).
- 2 **Construtor** [2,0 pontos] — recebe dois números inteiros; inicializa `this` de forma que o coeficiente `a` receba o 1º parâmetro e o coeficiente `b` receba o 2º parâmetro.
- 3 **mais** [2,0 pontos] — `public Complexo mais(Complexo outro)` — produz e retorna nova instância que expressa a soma de `this` com o parâmetro recebido.
- 4 **menos** [2,0 pontos] — `public Complexo menos(Complexo outro)` — retorna nova instância que expressa a diferença (`this` menos o parâmetro).
- 5 **vezes** [2,0 pontos] — `public Complexo vezes(Complexo outro)` — retorna nova instância que expressa a multiplicação (`this` pelo parâmetro).
- 6 **divididoPor** [2,0 pontos] — `public Complexo divididoPor(Complexo outro)` — retorna nova instância que expressa a divisão (`this` pelo parâmetro).
- 7 **Métodos padrão** — a critério, implementar `toString`, `equals`, `hashCode`, `compareTo` e `clone` (com construtor de cópia auxiliar), de acordo com necessidade e aplicabilidade.

Critério de correção do item (g) — atenção especial

- desconto de 1,5 ponto por método feito **sem** necessidade/aplicabilidade;
- desconto de 1,5 ponto por método **não feito** quando havia necessidade/aplicabilidade;
- desconto de 1,5 ponto por método feito **errado** ou fora do padrão ensinado, quando havia necessidade/aplicabilidade.

Obs. geral: quaisquer erros devem ser sinalizados com lançamento de exceções.

Padrão comum aos 5 exercícios: classe composta (usa outras classes/valores como atributos) → construtor que valida uma restrição de negócio → métodos de operação que recebem outra instância e retornam uma nova instância, ou métodos que calculam algo a partir da diferença entre dois pontos → decisão criteriosa de quais métodos padrão (`toString`, `equals`, `hashCode`, `compareTo`, `clone`+cópia) fazem sentido.